

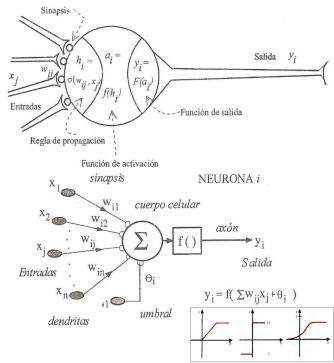
T8- Sistemas conexonistas alimentados hacia adelante

La neurona artificial

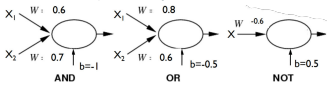
Concepto y arquitectura matemática

La neurona artificial (o Elemento de Procesado, EP) es la unidad base de las redes y su funcionamiento consta de las siguientes fases:

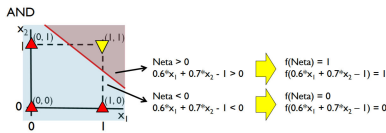
- Entradas y Pesos: Recibe señales de entrada (x_j) que se multiplican por unos pesos sinápticos (w_{-ij}). Las conexiones pueden ser excitatorias ($w_{-ij} > 0$), inhibitorias ($w_{-ij} < 0$) o inexistentes ($w_{-ij} = 0$). Los pesos son fundamentales porque codifican el conocimiento de la red.
- Sumatorio y Bias: Se realiza un sumatorio de todas las entradas ponderadas. A este resultado se le añade un Umbral o Bias (b_{-i} o θ_i), que actúa como una predisposición constante de la neurona a activarse. Esto forma la 'Entrada Neta'.
- Función de Transferencia: A la entrada neta se le aplica una función de activación (FO) para generar la salida final de la neurona (y_i).



Emulación de puertas lógicas



Modificando cuidadosamente los valores de los pesos y el bias, una sola neurona puede comportarse como puertas lógicas básicas (AND, OR, NOT). Visualmente, la neurona crea una frontera de decisión (una recta definida por Neta = 0) que divide el espacio. Por un lado de la recta (Neta > 0) la salida es 1 y por el otro (Neta < 0) la salida es 0.



Concepto de aprendizaje

Para que la red dé la salida deseada, hay que modificar sus pesos y bias. Este proceso se llama entrenamiento, y requiere un conjunto de "patrones" o ejemplos, consistentes en pares de (Entradas, Salida deseada).

Modelo ADALINE

ADALINE (Adaptive Linear Element): Es un modelo básico de RNA caracterizado por tener una función de transferencia estrictamente lineal (función purelin).

Regla Delta o Método LMS

Propuesta por Widrow y Hoff (1960), la regla LMS (Least Mean Square o Regla Delta) es el algoritmo para entrenar al ADALINE. Es un aprendizaje supervisado por corrección de error. El objetivo matemático es minimizar la Función de Coste, que es el Error Cuadrático Medio (ECM) sobre todos los ejemplos de entrenamiento.

Error para el ejemplo k:

- $E_k = d_k - y_k$ (salida deseada menos salida obtenida)

Error cuadrático (para el ejemplo k):

- Función de pérdida (loss function)

$$E_k = \frac{1}{2} (d_k - y_k)^2$$

siendo d_k la salida deseada y y_k la salida obtenida.

Error cuadrático medio (para todos los ejemplos):

- Función de coste (cost function) o pérdida

$$E = \sum_{k=1}^L E_k = \frac{1}{2} \sum_{k=1}^L (d_k - y_k)^2$$

Gradiente descendente

Para minimizar el error, la red calcula la derivada del error respecto a los pesos (esto nos da la pendiente o gradiente).

Regla de desplazamiento: * Si el peso está lejos del error mínimo (pendiente muy inclinada), la modificación del peso es grande.

* Si está cerca del mínimo (pendiente casi plana), la modificación es pequeña.

Ecuación de actualización: Tras derivar usando la regla de la cadena, el peso en el instante de tiempo $t+1$ se actualiza con la fórmula: $w_j(t+1) = w_j(t) + \mu \cdot (d_k - y_k) \cdot x_{jk}$ (Donde μ es la tasa de velocidad de aprendizaje).

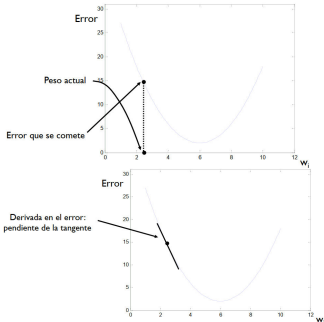
Control de convergencia

La Tasa de aprendizaje (μ) es crítica.

* Si μ es muy alto, los saltos son tan grandes que la red puede pasarse el mínimo de largo y no converger nunca.

Si μ es muy bajo, el descenso es seguro, pero el entrenamiento será excesivamente lento.

Pasos del algoritmo: 1) Inicializar pesos al azar. 2) Aplicar entradas. 3) Computar salida. 4) Calcular error. 5) Actualizar pesos. 6) Repetir hasta que el ECM sea aceptable.



Principal desventaja: Solo puede resolver problemas y aproximaciones que sean estrictamente lineales. Se utiliza hoy en día fundamentalmente en procesamiento de señales y filtros de ruido.

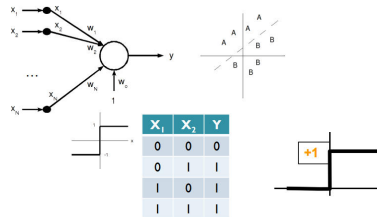
El perceptrón y sus limitaciones

Características básicas

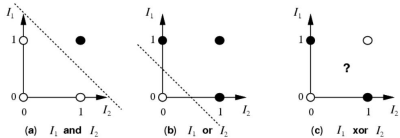
Desarrollado por Rosenblatt en 1958 (anterior al ADALINE).

Estructura: Tiene una capa de entrada y un único elemento (neurona) en la capa de salida.

Diferencia con ADALINE: Utiliza una función de transferencia umbral (hardlimiter o escalón) en lugar de una función puramente lineal. Devuelve salidas discretas (ej. 0 o 1).



Capacidad de representación y el problema del XOR



El perceptrón monocapa tiene una capacidad muy limitada: solo es capaz de resolver problemas que sean linealmente separables (donde las clases se pueden dividir por una recta en 2D, un plano en 3D, o un hiperplano en N dimensiones).

Un ejemplo que sí puede resolver es la función lógica OR y la función AND.

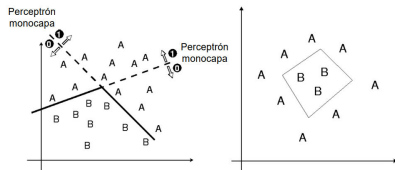
El problema trampa (XOR): Los ejemplos de la función XOR (o exclusivo) no se pueden separar mediante una sola línea recta. Esto demostró que un Perceptrón monocapa nunca podría solucionar el problema del XOR.

El perceptrón multicapa (MLP) y su arquitectura

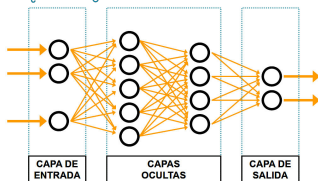
Solución al XOR

Para resolver problemas no linealmente separables (como el XOR), la solución es añadir más capas, creando el Perceptrón Multicapa (MLP).

¿Cómo funciona visualmente? La primera capa oculta traza varias líneas (crea semiespacios), la segunda capa une esos semiespacios creando regiones, y la capa de salida agrupa las regiones que pertenecen a la misma clase.



Arquitectura general de MLP



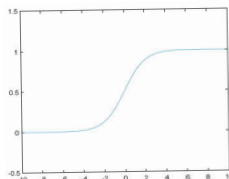
- Capa de entrada: Sus neuronas no computan nada; solo almacenan las entradas y las pasan a la siguiente capa. Hay una neurona por cada variable de entrada.
- Capa(s) oculta(s): Procesan la información. No existe una regla fija para saber cuántas capas o neuronas poner (se decide por ensayo y error, aunque no se recomiendan más de dos capas ocultas).
- Capa de salida: Emiten el resultado final. Hay una neurona por cada salida deseada.
- El Conocimiento: Reside en los pesos de las conexiones y el bias. Es un conocimiento distribuido, lo que otorga autoorganización y tolerancia a fallos, pero como desventaja hace imposible explicar su razonamiento interno (Caja negra).

Funciones de transferencia y aproximación universal

Se asume que todas las neuronas de una misma capa comparten la misma función de transferencia. No están limitadas a funciones lineales o umbral.

Funciones típicas:

- Escalón/Umbral: Para clasificación pura (0/1 o -1/1).
- Lineal: Para problemas de regresión o ajuste de curvas.
- Logarítmica Sigmoidal (logsig): Salidas acotadas entre 0 y 1.
- Tangente Sigmoidal Hiperbólica (tansig): Salidas acotadas entre -1 y 1.



La función de transferencia de las capas ocultas NO PUEDE ser lineal. Si lo fuera, la red perdería toda su capacidad de representación no lineal. En la capa de salida dependerá del problema (lineal para regresión, logsig/softmax para clasificación).

Teoremas de Aproximación Universal: Demostraron que un MLP con suficientes unidades ocultas no lineales puede aprender cualquier función o relación continua.

El algoritmo de backpropagation

¿Por qué no usar la Regla Delta sencilla? La regla Delta requiere conocer la "salida deseada" para calcular el error. En el MLP, conocemos la salida deseada final, pero no sabemos cuál debería ser la salida ideal de las neuronas ocultas. Por tanto, no podemos calcular su error directamente. Solución: La Regla Delta Generalizada o Backpropagation. Es un aprendizaje supervisado off-line por corrección de error.

Mecanismo:

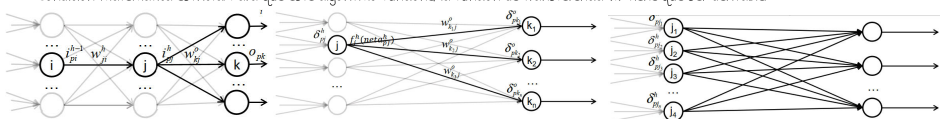
$$\delta_{pk} = (y_{pk} - o_{pk})$$

1. Se calcula el error en la capa de salida.

2. Ese error se "propaga hacia atrás" (retropropagación) hacia las capas ocultas usando la regla de la cadena de las derivadas.

3. Los términos de error de las capas ocultas dependen matemáticamente de los términos de error de la capa de salida conectados a ellas.

Condición matemática estricta: Para que este algoritmo funcione, la función de transferencia (f) tiene que ser derivable.



Control de convergencia, gradiente y mínimos

Para minimizar el error, la red desciende por una superficie matemática multidimensional (Descenso de Gradiente) ajustando los pesos en función de la Tasa o velocidad de aprendizaje (μ).

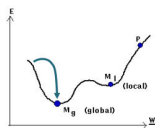
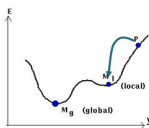
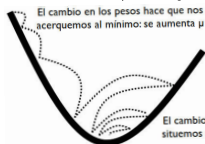
- Si μ es alto: Los incrementos de los pesos son muy grandes. Riesgo de saltarse el mínimo y "rebotar" por las paredes del valle sin asentar la solución.

- Si μ es bajo: Movimientos muy seguros pero excesivamente lentos para llegar al fondo.

- Mínimos Locales vs. Globales: El algoritmo busca el fondo del valle (gradiente = 0). Sin embargo, una superficie puede tener "baches" intermedios.

- El algoritmo puede estancarse en un Mínimo Local y parar el aprendizaje creyendo que ha terminado, aunque el error siga siendo demasiado alto.

- Al iniciar los pesos de forma aleatoria, (cada vez que entrenas una red da un resultado distinto). Puede que en un intento caiga en un mínimo local y en el siguiente encuentre el Mínimo Global.



1. Entrenar de nuevo (gracias a la iniciación aleatoria, puede que en el segundo intento encuentre el mínimo global)

2. Modificar la tasa de aprendizaje (μ).

3. Cambiar la arquitectura (añadir más capas o neuronas).

Conjunto de datos y evaluación

Para saber si una red está realmente bien entrenada y es capaz de generalizar (comportarse bien ante datos que nunca ha visto), los datos se dividen en grupos. Las diapositivas abarcan inicialmente dos conceptualizaciones muy importantes para el examen:

- Conjunto de Entrenamiento: Son los datos que guían el proceso de cálculo de los pesos. La red "aprende" minimizando el error sobre estos datos.
- Conjunto de Test (Evaluación final): Es un conjunto de datos nuevos que no interviene en el entrenamiento en absoluto. El error que obtenga la red en este conjunto de Test es el que dicta realmente su validez y su capacidad de generalización real.

Sobreentrenamiento

Capacidad de generalización

Definición: Es la capacidad de la red para responder correctamente ante patrones que no se han utilizado durante el entrenamiento.

Objetivo Real: No queremos que la red "memorice" los ejemplos (mapeo exacto), sino que aprenda la regla subyacente que los genera.

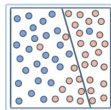
El dilema: Si la red es demasiado simple, no aprende el problema. Si es demasiado compleja (muchas neuronas/capas), tiene "demasiada memoria" y tiende a memorizar el ruido de los datos en lugar de la tendencia.

¿Qué es sobreentrenamiento?

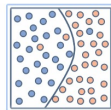
Ocurre cuando la red se ajusta tanto a los datos de entrenamiento que pierde su capacidad de generalización.

- Síntoma en el test: El error en el conjunto de entrenamiento es muy bajo (o casi cero), pero el error en el conjunto de test es muy alto.

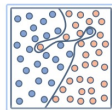
- Causa: Se ha entrenado durante demasiadas épocas o la red tiene una complejidad excesiva para la cantidad de datos disponibles. La red "aprende" incluso el ruido o los errores específicos de los ejemplos de entrenamiento.



Underfitting



Optimal



Overfitting

Conjunto de validación

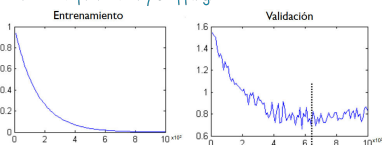
Para detectar el sobreentrenamiento antes de que ocurra, dividimos los datos no en dos, sino en tres grupos:

- Entrenamiento: Para ajustar los pesos.

- Validación: Para monitorizar la generalización durante el proceso de entrenamiento. (Estos datos no ajustan los pesos, pero sí sirven para decidir cuándo parar).

- Test: Solo para la evaluación final una vez terminado todo el proceso.

Parada temprana (Early stopping)



Es la técnica más común para evitar el sobreentrenamiento.

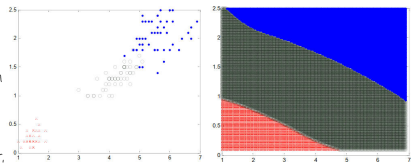
Funcionamiento:

1. Se entrena la red y se calcula el error tanto en Entrenamiento como en Validación en cada época.
2. Al principio, ambos errores bajan.
3. Llega un punto en que el error de Entrenamiento sigue bajando, pero el de Validación empieza a subir.
4. Acción: El entrenamiento debe detenerse en ese punto mínimo del error de validación, ya que a partir de ahí empieza el sobreentrenamiento.

Validación cruzada (Cross-Validation)

Se usa cuando tenemos pocos datos y no podemos permitirnos dejar muchos fuera para validación.

- K-Fold Cross-Validation: * Se dividen los datos en K bloques (ej. 10 bloques).
 - Se entrena K veces: en cada vez, se usa un bloque distinto como validación y el resto como entrenamiento.
 - El resultado final es el promedio de los errores obtenidos en las K ejecuciones.
- Ventaja: Permite usar todos los datos tanto para entrenar como para validar, asegurando que el modelo sea robusto y no dependa de una división "afortunada" o "desafortunada" de los datos.



Aplicaciones de las RRNNAA

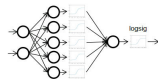
Una vez superada la teoría matemática, el tema aborda para qué sirven exactamente estas arquitecturas en la práctica. Las Redes Neuronales son excepcionalmente buenas en tareas donde los algoritmos tradicionales fallan (aquellas que requieren intuición o reconocimiento de patrones complejos).

Principales áreas de aplicación:

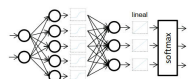
- Reconocimiento de Patrones y Visión Artificial: Identificación de caracteres manuscritos, reconocimiento facial, o análisis de imágenes médicas (aunque con las restricciones que veremos en las conclusiones).
- Procesamiento de Señales: Como vimos con el ADALINE, son excelentes para filtrar ruido en líneas de comunicación o procesar señales de audio y voz.
- Control de Sistemas y Robótica: Sistemas de navegación autónoma, brazos robóticos y control de trayectorias donde el entorno es dinámico
- Predicción y Análisis Financiero: Previsión de series temporales, predicción de la bolsa o evaluación de riesgos crediticios.

Clasificación:

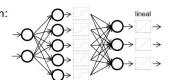
Dos clases:



Más de dos clases:



Regresión:



Integración en Sistemas existentes (Hardware / Software)

Un aspecto muy destacado en la recta final de las diapositivas es la facilidad de inserción de las RRNNAA para aplicaciones específicas dentro de sistemas que ya existen.

Implementación en Hardware: Se pueden integrar físicamente en chips dedicados.

Desarrollo Incremental: Esta modularidad permite que las redes neuronales se utilicen para mejorar sistemas informáticos o industriales de forma incremental. La gran ventaja de esto es que cada paso puede ser evaluado individualmente antes de acometer un desarrollo o una inversión más amplia.

Conclusiones

Ventajas del uso de RRNNAA

1. **Aprendizaje adaptativo:** Capacidad de aprender a realizar tareas concretas basándose en un entrenamiento o en una experiencia inicial.
2. **Autoorganización:** La red neuronal puede crear su propia organización o representación interna de la información que recibe durante la etapa de aprendizaje (¡Este era el punto que faltaba!)
3. **Tolerancia a fallos:** Al tener el conocimiento distribuido en toda su estructura, la destrucción o fallo parcial de la red no provoca un colapso total sino que algunas capacidades se retienen y el sistema sufre una degradación progresiva (grácil).
4. **Capacidad de generalización:** Ante la entrada de datos nuevos (que no estaban en su conjunto de entrenamiento), la red es capaz de producir resultados coherentes y generalizar la tendencia del problema.
5. **Operación en tiempo real:** Gracias a que su arquitectura permite un procesamiento en paralelo, los cálculos en la fase de ejecución se pueden realizar a altísima velocidad.
6. **Fácil inserción dentro de la tecnología existente:** Su modularidad permite incorporarlas de forma sencilla y económica dentro de sistemas actuales, existiendo incluso chips especializados (hardware) para este propósito.

Desventajas del uso de RRNNAA

1. Implementación deficiente como sistemas paralelos

Las redes neuronales son sistemas inherentemente paralelos (muchas neuronas procesando a la vez). Sin embargo, se suelen implementar en máquinas en serie (nuestros ordenadores tradicionales basados en la arquitectura Von Neumann), donde solo se ejecuta una instrucción a la vez. Consecuencia: Modelar procesos paralelos en máquinas en serie consume muchísimo tiempo y recursos computacionales.

2. No existencia de una regla fija para construir una RNA

Para un problema dado, no hay un manual exacto. Hay muchos factores empíricos a tener en cuenta: qué algoritmo de aprendizaje usar, la arquitectura óptima, el número de neuronas por capa, cuántas capas poner, o cómo representar los datos. Todo se basa en la prueba y error.

3. El problema de la "caja negra" e imposibilidad de explicación

Una RNA funciona como una caja negra: introduces unas entradas y te da unas salidas, pero no es posible comprender la relación matemática exacta entre ellas de forma humana.

Contraste: En un modelo estadístico tradicional puedes ver qué variables forman parte del modelo y qué peso específico tiene cada una en la decisión final.

El límite médico: Esta incapacidad de explicar su razonamiento (falta de trazabilidad) provoca que no se puedan utilizar de forma autónoma en entornos críticos como la medicina, donde un profesional necesita saber por qué el sistema ha emitido un diagnóstico concreto.

4. Dependencia masiva de los datos

Existe una regla directamente proporcional: cuanto más flexible y compleja se requiera que sea una RNA, más información habrá que enseñarle. Esto significa que las redes más potentes necesitan unas bases de datos gigantescas (más patrones de entrenamiento) para no caer en errores o memorizar los datos (sobreentrenamiento).